

Make `std::ignore` a first-class object

Document #: P2968R1
Date: 2023-11-07
Project: Programming Language C++
Audience: WG21 - Library & Library Evolution
Reply-to: Peter Sommerlad
<peter.cpp@sommerlad.ch>

Contents

- [1 Abstract](#)
- [2 History](#)
 - [2.1 R1 incorporating some discussion](#)
 - [2.2 R0 initial revision](#)
- [3 Introduction](#)
 - [3.1 Non-tuple applications of `std::ignore`](#)
 - [3.2 LWG issue 2933](#)
 - [3.3 LWG issue 3978](#)
- [4 Mailing List discussions](#)
- [5 Questions to LEWG](#)
- [6 Questions to LWG](#)
- [7 Impact on existing code](#)
- [8 Wording](#)
- [9 References](#)

§ 1 Abstract

All major open source C++ library implementations provide a suitably implemented `std::ignore` allowing a no-op assignment from any object type. However, according to some C++ experts the standard doesn't bless its use beyond `std::tie`. This paper also resolves issue [lwg2933](#) by proposing to make the use of `std::ignore` as the left-hand operand of an assignment expression official.

§ 2 History

§ 2.1 R1 incorporating some discussion

- mention bit-fields and suggest that they cannot be a valid RHS to `std::ignore`'s assignment, addresses [LWG 3978](#). Add question if RHS non-volatile bit-fields should be supported at all.
- Incorporate (most of) Arthur O'Dwyer's input and suggestions, including his analysis of existing major implementations.
- recommend spec as code using `operator=(auto&&)` preventing bit-fields as rhs, to prevent accidental side-effects through volatile bit-fields.

§ 2.2 R0 initial revision

- Addresses [LWG 2933](#)
- Takes up some mailing list discussion.

§ 3 Introduction

All major C++ open source standard libraries provide similar implementations for the type of `std::ignore`. However, the semantics of `std::ignore`, while useful beyond, are only specified in the context of `std::tie`.

§ 3.1 Non-tuple applications of `std::ignore`

Programming guidelines for C++ safety-critical systems consider all named functions with a non-void return type similar to having the attribute `[[nodiscard]]`.

As of today, the means to disable diagnostic is a static cast to void spelled with a C-style cast `(void) foo();`. This provides a teachability issue, because, while C-style casts are banned, such a C-style cast need to be taught. None of the guidelines I am aware of, dared to ask for a `static_cast<void>( foo() );` in those situation.

With the semantics provided by the major standard library implementations and the semantics of the example implementation given in the [cppreference.com](#) site, it would be much more documenting intent to write

```
std::ignore = foo();
```

instead of the C-style void-cast.

To summarize the proposed change:

1. better self-documenting code telling the intent

2. Improved teachability of C++ to would-be C++ programmers in safety-critical environments

§ 3.2 LWG issue 2933

This issue asks for a better specification of the type of `std::ignore` by saying that all constructors and assignment operators are `constexpr`. I don't know if that needs to be said that explicitly, but the assignment operator template that is used by all implementations should be mentioned as being `constexpr` and applicable to the global constant `std::ignore`.

§ 3.3 LWG issue 3978

This issue claims that the “no further effect” is not implementable when the rhs is a volatile bit-field. Providing a code-wise specification, we can eliminate the situation, because the rhs will not allow bit-fields (`auto&&`) or will allow only non-volatile bit-fields (`auto const&`).

§ 4 Mailing List discussions

After some initial drafts posted to `lib-ext@lists.isocpp.org` I got some further feedback on motivation and desire to move `ignore` to a separate header or utility:

Additional motivating usage by Arthur O'Dwyer:

```
struct DevNullIterator {
    using difference_type = int;
    auto& operator++() { return *this; }
    auto& operator++(int) { return *this; }
    auto operator*() const { return std::ignore; }
};

int a[100];
std::ranges::copy(a, a+100, DevNullIterator());
```

Giuseppe D'Angelo: As an extra, could it be possible to move `std::ignore` out of `<tuple>` and into `<utility>` or possibly its own header `<ignore>`?

Ville Voutilainen suggested a specification as code, such as:

```
// 22.4.5, tuple creation functions
struct ignore_type { // exposition only
    constexpr decltype(auto) operator=(auto&&) const & { return *this; }
};
inline constexpr ignore_type ignore;
```

or even more brief by me (the lvalue ref qualification is optional, but I put it here):

```
inline constexpr
struct { // exposition only
    constexpr decltype(auto) operator=(auto&&) const & noexcept { return
        *this; }
} ignore;
```

Thanks to Arthur O’Dwyer for repeating my analysis of the three major open source library implementations.

Note that using `auto&&` instead of `auto const &` prevents the use of a bit-field on the right-hand side of the assignment. Here exists implementation divergence between the different libraries.

§ 5 Questions to LEWG

Since there was the request to make `std::ignore` available without having to include `<tuple>` the following questions are for LEWG. Note, any yes will require a change/addition to the provided wording and would also put some burden on implementors. An advantage might be slightly reduced compile times for users of `std::ignore` who do not need anything else from header `<tuple>`.

- Should `std::ignore` be made available via its own header `<ignore>` in addition to be available via `<tuple>`? Y/N (I would not suggest a new header.)
- If no: Should `std::ignore` be made available via header `<utility>` in addition to being available via `<tuple>`? Y/N (This makes sense in my opinion, `tuple_size` is there as well, for example)
- Should `std::ignore::operator=` support non-volatile bit-fields on the right hand side (`operator=(auto const&)`) or not (`operator=(auto&&)`) ?

The latter question is not an issue with existing code, because `tuple’s tie()` would never support bit-fields and there is actual implementation divergence between major vendors. I don’t have a strong opinion, but suppressing also non-volatile bit-fields helps in situation where a bit-field containing class mixes volatile and non-volatile members.

We have implementation divergence here (see below), which means we have implementation experience both ways:

- `libstdc++` and Microsoft both currently use `(const auto&)` (which supports non-volatile bit-fields (which are converted to rvalue implicitly) but not volatile bit-fields).
- `libc++` currently uses `(auto&&)` (which does not support bit-fields of any qualification). `libc++` has never received a bug report about its more-restrictive `std::ignore`, which is currently proposed.

§ 6 Questions to LWG

- Should the specification of the type of `std::ignore` use code or stick to a more generic text (see below)?
 - Note: just using text, can create an issue with using volatile (bitfields) as RHS, where the actual read on the volatile will happen. using `auto&&` as assignment operator's argument will prevent using a volatile bitfield on the RHS (according to [my experiments](#)).
- Do we need to mention “constexpr constructors” as asked for by LWG2933? (see parenthesized text in the Wording section) I don't think so.
- How would we mention availability from header `<utility>` ? (I prepared accordingly, thanks to Arthur's help)

§ 7 Impact on existing code

All three major standard library vendors (libstdc++, Microsoft, libc++) implement `std::ignore` in roughly the same way, but there are some minor differences that could be ironed out by this proposal.

```
// libstdc++
struct __ignore_t {
    constexpr const __ignore_t&
        operator=(const auto&) const
        { return *this; }
};
inline constexpr __ignore_t ignore{};

// Microsoft STL
struct __ignore_t {
    constexpr const __ignore_t&
        operator=(const auto&) const noexcept
        { return *this; }
};
inline constexpr __ignore_t ignore{};

// libc++
template<class T>
struct __ignore_t {
    constexpr const __ignore_t&
        operator=(auto&&) const
        { return *this; }
};
inline constexpr __ignore_t<unsigned char> ignore = __ignore_t<unsigned
    char>();
```

The difference between `operator=(const auto&)` and `operator=(auto&&)` is that the former will bind to non-volatile bit-field glvalues, whereas the latter will not. If we want to support non-volatile bit-fields, we should change the signature of `ignore::operator=` to take `(const auto&)` instead of `(auto&&)`. Neither signature binds to *volatile* bit-field glvalues; that is [LWG3978](#). Our wording makes `std::ignore = volatilebit.field;` obviously ill-formed, which resolves [LWG3978](#).

The difference between `noexcept` and `not` is negligible; we could mandate that `std::ignore::operator=` be `noexcept`, or we could leave it non-`noexcept`. The vendor is already permitted to strengthen the `noexceptness` of any standard library function; Microsoft takes advantage of this freedom already. We suggest that as long as we're touching this area anyway, we should go ahead and add the `noexcept`.

The programmer can detect whether `ignore`'s type is a template. We suggest either mandating that `ignore`'s type not be a template, or finding a way to leave it unspecified.

However, may be LEWG will decide on being more specific and less hand-wavy on the semantics of the type underlying `std::ignore` and even follow the suggestion to move its definition into another header (`<utility>` or `<ignore>`).

The mailing list discussion seems to favor making it available through `<utility>` as well to `<tuple>`, as is `std::tuple_size`, for example.

§ 8 Wording

In [tuple.syn](#) change the type of `ignore` from “unspecified” to an exposition-only type

```
+// [utility.ignore], ignore
+struct ignore_t { // exposition only
+    constexpr const ignore_t&
+        operator=(auto&&) const & noexcept
+        { return *this; }
+};
+inline constexpr ignore_type ignore;
// 22.4.5, tuple creation functions
-inline constexpr unspecified ignore;
```

In [tuple.creation](#), remove the normative text about `ignore`:

```
template<class... TTypes>
    constexpr tuple<TTypes&...> tie(TTypes&... t) noexcept;
5. Returns: tuple<TTypes&...>(t...).
-When an argument in t is ignore, assigning any value to the corresponding
  tuple element has no effect.
6. [Example 2: tie functions allow one to create tuples that unpack
  tuples into variables.
ignore can be used for elements that are not needed:
    int i; std::string s;
    tie(i, ignore, s) = make_tuple(42, 3.14, "C++");
    // i == 42, s == "C++"
- end example]
```

In [utility.syn](#), add `ignore` to the `<utility>` header also:

```
template<class... T>
    using index_sequence_for = make_index_sequence<sizeof...(T)>;

+// [utility.ignore], ignore
+struct ignore_t { // exposition only
```

```
+ constexpr const ignore_t&
+   operator=(auto&&) const & noexcept
+   { return *this; }
+};
+inline constexpr ignore_t ignore;

// [pairs], class template pair
template<class T1, class T2>
    struct pair;
```

We don't think a feature-test macro is required for this patch, as it's just barely visible to the programmer, and there is no situation where the programmer would want to do something different based on the absence of the feature.

§ 9 References

- [LWG 2933](#)
- [LWG 3978](#)
- The markdown of this document is available at my [github](#)